

Data Apps with Streamlit and Its Siblings

From a Python script to a shared, governed app: Streamlit fundamentals, Streamlit in Snowflake, and where Gradio, Dash, Shiny and Databricks Apps fit

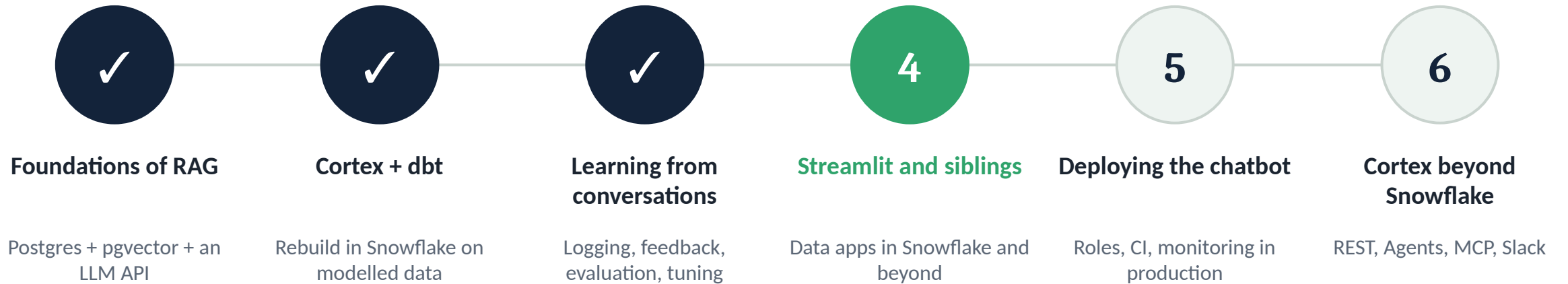


R. Firman Insan Muhammad (Vim)

SnowPro Certified • Data Engineering and AI Trainer

Scenario: Neo Nexus, a loyalty assistant for Caffè Nero. Illustrative only: all data, documents and figures are synthetic.

The assistant gets a user interface



Today's output: a Neo Nexus rewards explorer and a chat shell with feedback, running in Streamlit in Snowflake and shared by role.

By the end of today you can...



Explain

Streamlit's rerun execution model, and use session state and caching correctly because of it.



Build

data apps and a chat interface with feedback, using widgets, layout and chat elements.



Run

apps inside Snowflake, choose a runtime, and reason about owner's rights and sharing by role.



Choose

between Streamlit, Gradio, Dash, Shiny and hosting platforms such as Databricks Apps, with reasons.

The last mile between a dashboard and a product

BI dashboard

Power BI, Tableau, Looker

Read-only views of modelled data; great for monitoring, poor at input, logic and conversation.

Data app

Streamlit, Gradio, Dash, Shiny

Python-only, interactive, can call models and write back. Built by data teams in days.

Custom web app

React + an API, mobile apps

Full control and scale for customer-facing products; needs a front-end team and months.

faster, simpler

more control, more effort

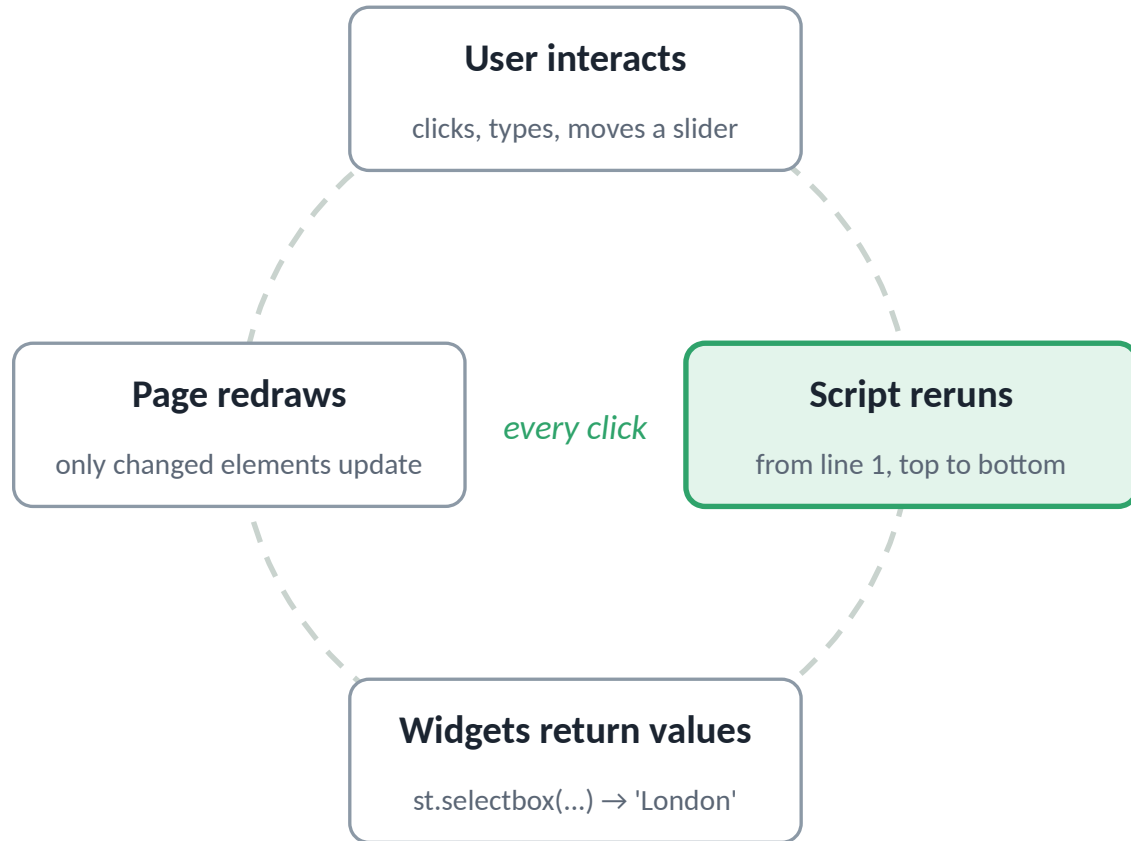
Our assistant is a data app: internal users, Python team, data in the warehouse.

01

Streamlit fundamentals

The rerun model, widgets and layout, state and caching, and the chat pattern

Every interaction reruns the whole script, top to bottom



Consequence 1 • State

Variables reset on every rerun. Anything that must persist, such as chat history, lives in st.session_state.

Consequence 2 • Caching

Expensive work runs on every click unless cached with st.cache_data or st.cache_resource.

The payoff

Code reads like a plain script: no callbacks, no front-end, no HTML.

A working data app in about fifteen lines

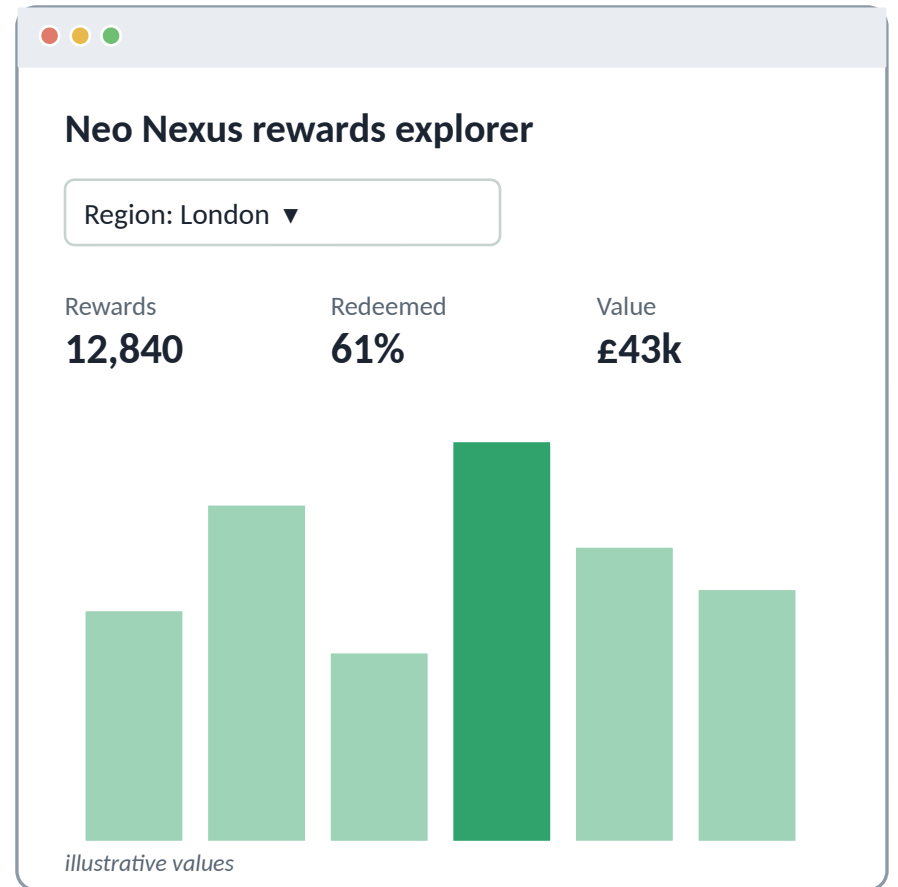
APP.PY

```
import streamlit as st

st.title("Neo Nexus rewards explorer")
region = st.selectbox("Region", ["London", "Manchester", "Edinburgh"])

@st.cache_data(ttl=600)                                # rerun-safe: cached
def load_rewards(region):
    return session.sql(
        "select * from marts.fct_rewards f "
        "join marts.dim_store p using (store_id) "
        "where p.region = ?", params=[region]
    ).to_pandas()

df = load_rewards(region)
c1, c2 = st.columns(2)
c1.metric("Rewards", len(df))
c2.metric("Redeemed", f"{{(df.REWARD_STATUS == 'Redeemed').mean():.1%}}")
st.bar_chart(df.groupby("ISSUED_MONTH").size())
```



Four families of elements cover most apps



Input

text_input, selectbox, slider, date_input, file_uploader, form



Display

dataframe, metric, bar_chart, line_chart, map, markdown



Layout

columns, tabs, sidebar, expander, container, dialog



Chat and feedback

chat_message, chat_input, write_stream, feedback

Tip: wrap related inputs in `st.form` so the script reruns once on submit, not on every keystroke.

Remember with session state; avoid repeat work with caching

PYTHON

```
# per-user memory that survives reruns
if "messages" not in st.session_state:
    st.session_state.messages = []

# results of a query: copied, refreshed after ttl
@st.cache_data(ttl=600)
def rewards_by_month(region):
    return session.sql(...).to_pandas()

# one shared object for all users: not copied
@st.cache_resource
def search_service():
    return (Root(session).databases["NEO_NEXUS"]
            .schemas["SERVING"]
            .cortex_search_services["LOYALTY_SEARCH"])
```

Use	For	Example
st.session_state	per-user values across reruns	chat history, selected filters
st.cache_data	serialisable results, with ttl	query results, API responses
st.cache_resource	shared, unserialisable objects	sessions, service handles, models

Classic mistakes: caching a connection with `cache_data`, or data with `cache_resource`.

A chat interface with feedback in about twenty lines

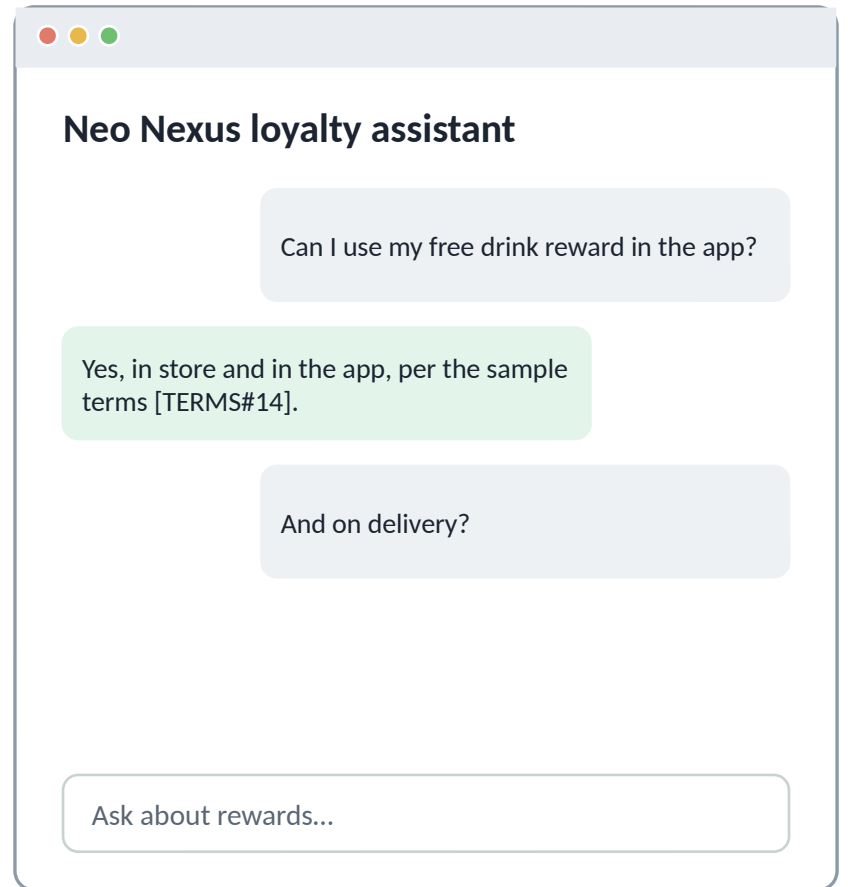
CHAT.PY

```
if "messages" not in st.session_state:
    st.session_state.messages = []

for m in st.session_state.messages:          # replay history
    st.chat_message(m["role"]).write(m["text"])

if q := st.chat_input("Ask about rewards"):
    st.chat_message("user").write(q)
    text, turn_id = answer(q, st.session_state.messages)
    st.chat_message("assistant").write(text)
    st.session_state.messages += [
        {"role": "user", "text": q},
        {"role": "assistant", "text": text, "turn_id": turn_id}]

last = st.session_state.messages[-1:] or [{}]
if tid := last[0].get("turn_id"):
    score = st.feedback("thumbs", key=f"fb_{tid}")
    if score is not None:                      # 1 = up, 0 = down
        save_feedback(tid, 1 if score == 1 else -1)
```

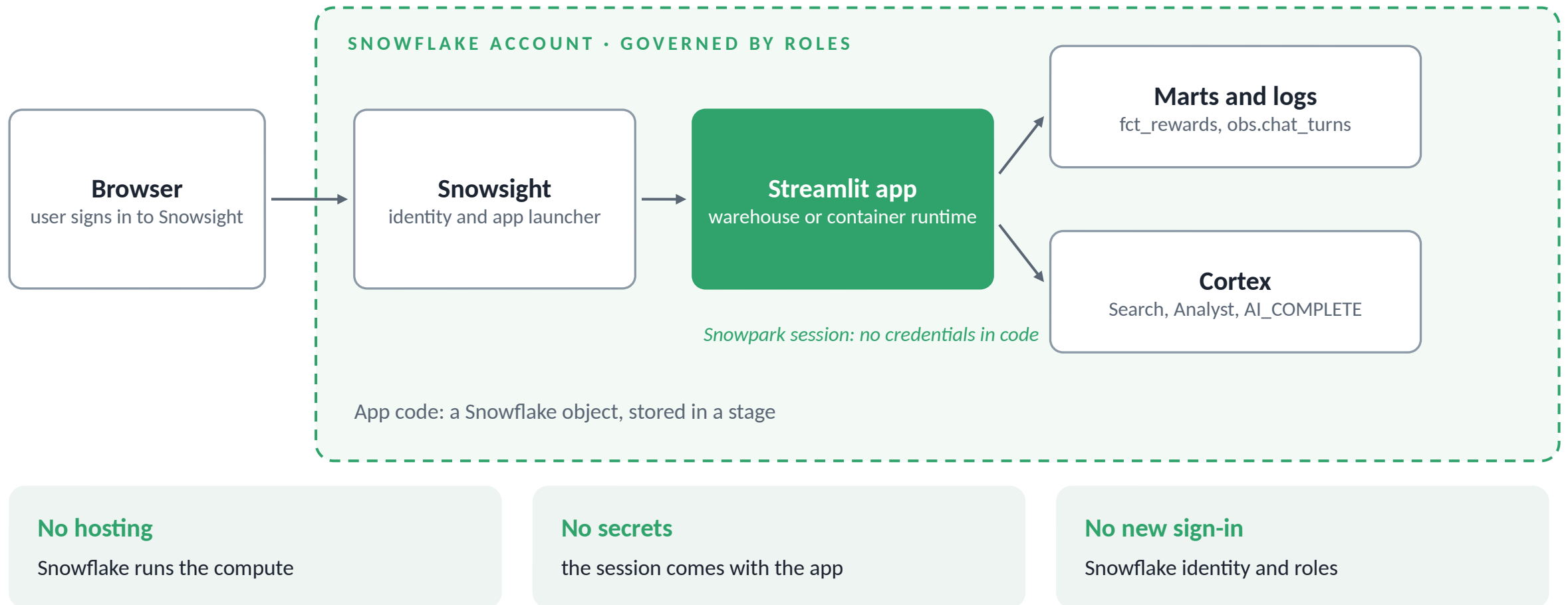


02

Streamlit in Snowflake

The same framework, running next to the data: sessions, runtimes, deployment and owner's rights

The app runs inside the platform that holds the data



Develop locally, run in Snowflake, change nothing

PYTHON • PORTABLE SESSION

```
@st.cache_resource
def get_session():
    try:
        # inside Snowflake
        from snowflake.snowpark.context import get_active_session
        return get_active_session()
    except Exception:
        # local development
        return st.connection("snowflake").session()

session = get_session()
```

LOCAL CONFIG

```
# .streamlit/secrets.toml (local only, never committed)
[connections.snowflake]
account = "..." user = "..." authenticator = "externalbrowser"
```

In Snowflake

`get_active_session()` returns the app's session. No credentials exist in the app at all.

On a laptop

`st.connection` reads `secrets.toml`. Use SSO or key-pair auth, and keep the file out of Git.

Warehouse runtime or container runtime

	Warehouse runtime	Container runtime
Compute	A virtual warehouse	A compute pool (Snowpark Container Services)
Packages	Snowflake's curated Anaconda channel	PyPI, via an external access integration
Set with	QUERY_WAREHOUSE	RUNTIME_NAME + COMPUTE_POOL
Good for	Simple apps, standard packages	Newer or niche packages, heavier apps

Either way: the SQL your app runs uses QUERY_WAREHOUSE, so size that warehouse for the queries, not the UI.

An app is a database object: create it, grant it

SNOWFLAKE SQL

```
-- upload app files to a stage, then:
create or replace streamlit apps.neo_nexus_explorer
  from @apps.code_stage/explorer
  main_file      = 'app.py'
  query_warehouse = app_wh
  title          = 'Neo Nexus rewards explorer';

-- container runtime variant adds:
--   runtime_name = 'SYSTEM$ST_CONTAINER_RUNTIME_PY3_11'
--   compute_pool = app_pool

-- share with CRM analysts
grant usage on streamlit apps.neo_nexus_explorer
  to role crm_analyst;
```

Code in a stage

Versioned alongside the rest of the project.

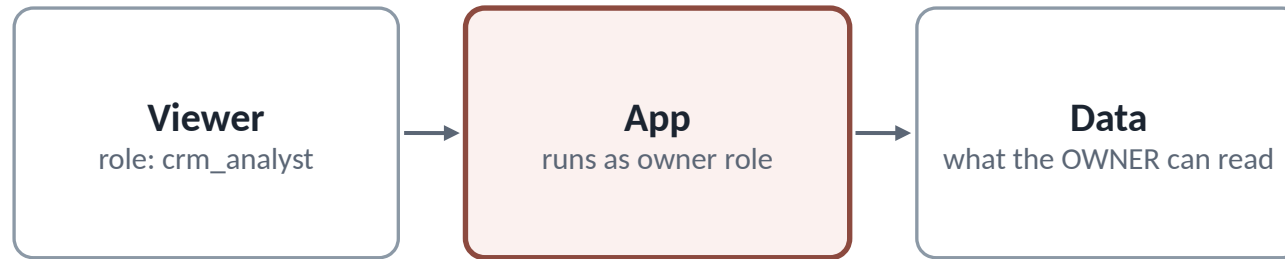
Shared by role

USAGE on the app; no public URL.

Automated in Module 5

Snowflake CLI and CI deploy the same objects.

Apps run with the owner's rights: design for it



The viewer's own grants do not limit what the app's queries can read.

So design for it

Give each app a dedicated owner role with least privilege: marts and the search service only. Keep sensitive columns masked for that role.

Different users, different rows?

Filter explicitly on the viewer's identity inside the app, and test it with two real users.

Audit

Queries appear in query history under the app's owner; log the viewer yourself in obs.chat_turns.

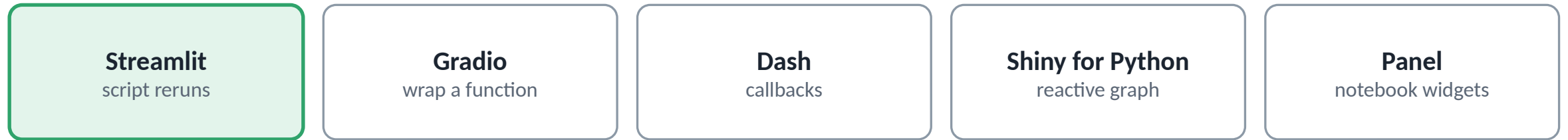
03

The family

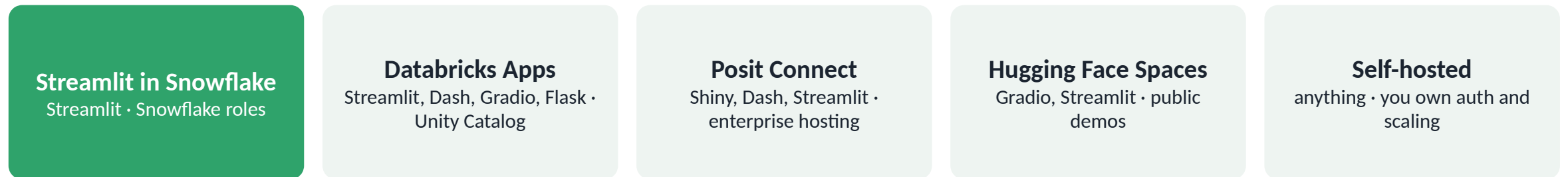
Gradio, Dash, Shiny, Panel and the platforms that host them: same problem, different trade-offs

Frameworks and platforms are separate choices

FRAMEWORKS • how you write it



PLATFORMS • where it runs and who can open it



Decide twice: the framework fits your team and interaction style; the platform fits your data, identity and governance.

Same problem, five execution models

Framework	Execution model	Strongest at	Watch out for
Streamlit	Whole script reruns on each interaction	Fastest path from script to data app; chat elements	Rerun cost in large apps
Gradio	Wraps functions; event handlers on components	Model demos, chat UIs, Hugging Face Spaces	Less suited to dashboards
Dash	Explicit callbacks, inputs to outputs	Complex, multi-page analytical apps	More boilerplate
Shiny for Python	Reactive graph: only dependants recompute	Efficient analytics; R teams moving to Python	Smaller Python ecosystem
Panel	Reactive widgets, notebook-first	Apps grown from Jupyter notebooks	Steeper styling curve

All five are open source Python frameworks; all can build a chat assistant.

The same assistant front end in three frameworks

STREAMLIT • 6 LINES

```
import streamlit as st

if q := st.chat_input("Ask"):
    st.chat_message("user") \
        .write(q)
    st.chat_message("assistant") \
        .write(answer(q))
```

GRADIO • 4 LINES

```
import gradio as gr

def respond(message, history):
    return answer(message)

gr.ChatInterface(respond).launch()
```

DASH • 14 LINES

```
from dash import (Dash, html, dcc,
                  Input, Output, State)

app = Dash()
app.layout = html.Div([
    dcc.Input(id="q"),
    html.Button("Ask", id="ask"),
    html.Div(id="a")]

@app.callback(Output("a", "children"),
              Input("ask", "n_clicks"),
              State("q", "value"),
              prevent_initial_call=True)
def respond(_, q):
    return answer(q)

app.run()
```

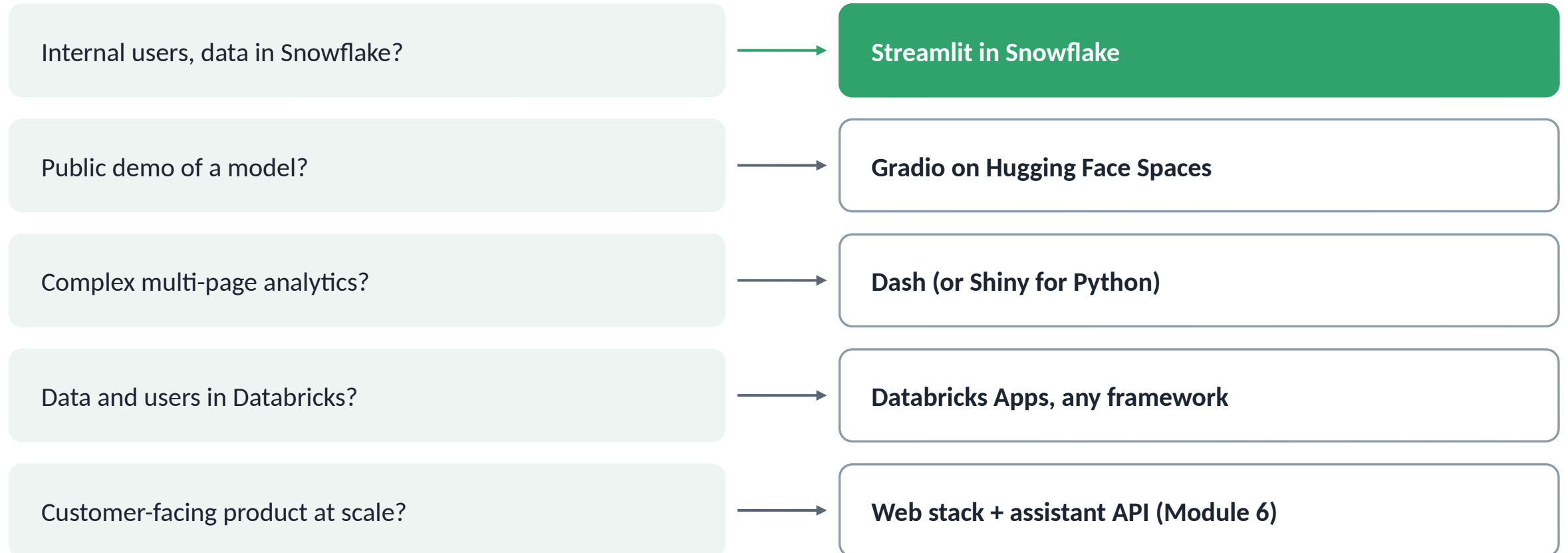
Same brain, different faces: keep `answer()` separate from the UI and you can switch frameworks in an afternoon.

Streamlit in Snowflake and Databricks Apps are close cousins

	Streamlit in Snowflake	Databricks Apps	Self-hosted
Frameworks	Streamlit	Streamlit, Dash, Gradio, Flask and more	Any
Identity	Snowflake users and roles	Databricks workspace identity	You build it (SSO, OAuth)
Data access	Snowpark session, no secrets	Unity Catalog, SQL warehouses	Credentials you manage
AI services	Cortex in the same account	Model Serving, Mosaic AI	Any API you call
Best when	Data and users live in Snowflake	Data and users live in Databricks	Public or customer-facing apps

Rule: host the app where the governed data already lives.

A short decision guide



The rest of this course: Streamlit in Snowflake.

Where Streamlit and Streamlit in Snowflake struggle



Reruns at scale

Large apps need caching, forms and fragments to stay fast.



Audience size

Built for internal users, not public, high-traffic sites.



Packages

Warehouse runtime uses the curated channel; PyPI needs the container runtime.



Outbound calls

External APIs need an external access integration.

Build the Neo Nexus explorer and chat shell

1

Local first

portable get_session(); run the rewards explorer on a laptop

15 min

2

Chat shell

chat_message, chat_input, session state; call answer()

20 min

3

Feedback

st.feedback writes to obs.feedback from Module 3

15 min

4

Deploy to Snowflake

stage the files; create streamlit; grant to a role

20 min

5

Owner's rights test

open as a narrower role; document what it can see

20 min

Deliverables

- Rewards explorer running in Streamlit in Snowflake
- Chat shell with history and thumbs feeding obs.feedback
- Shared with one role by grant, no URL sharing
- A note on what a narrower role could see, and why

Stretch: the same chat front end in Gradio, calling the same answer()

Questions worth arguing about

1. Why does Streamlit need both session state and caching, when other frameworks need neither in the same way?
2. An app runs with its owner's rights. How would you convince an auditor it is safe?
3. At what point should a data app be rebuilt as a proper web application?
4. How do you avoid being locked into one framework or one hosting platform?



References

Streamlit docs: Execution model; caching; session state; chat elements

Snowflake docs: About Streamlit in Snowflake; runtime environments

Snowflake docs: CREATE STREAMLIT; owner's rights and privileges

Databricks docs: Databricks Apps

Framework docs: Gradio ChatInterface; Dash callbacks; Shiny for Python; Panel

SnowPro prep: Streamlit in Snowflake topics in the current study guides

Deploying the chatbot



Structure the app

multi-page layout, config, secrets and environments



Ship with CI

Snowflake CLI, the Module 3 evaluation gate on every change



Run it in production

roles, monitoring, cost controls and rollback

Before Module 5: put your app code in a Git repository and install the Snowflake CLI.